

INF 345 — Fundamentals of DevOps

Lecture 2: Git & GitHub

Adil Akhmetov · Lesson 2



Recap — Lesson 1

- What does the **C** in CALMS stand for? (Culture)
- Name the three modules in this course's toolchain, in order. (Containers → Automation → CI/CD)
- Where does your Lab 01/02 grade actually come from? (RHA lab completion, not GitHub)

Today's agenda

- ☐ Why version control exists (the problem it solves)
- ☐ Git fundamentals: commits, diffs, history
- ☐ Branching & the GitHub workflow
- ☐ How you'll submit work this semester
- ☐ Where to learn more — curated resources
- ☐ Today's practice session

The scenario

```
report_final.docx , report_final_v2.docx ,  
report_final_v2_ACTUALLY_FINAL.docx ,  
report_final_v2_ACTUALLY_FINAL_fix.docx .
```

Two teammates edit the same file. One's changes silently overwrite the other's.
Nobody remembers which version is on the server.

Git exists to make this scenario impossible.

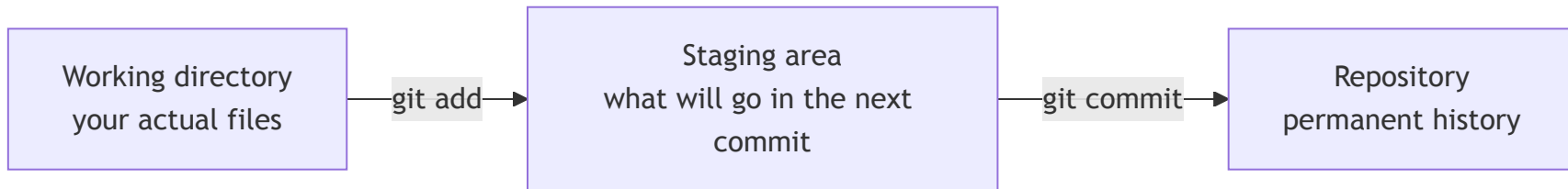
Block 1

Git fundamentals

Git tracks snapshots, not files

- Every commit is a **snapshot** of your whole project at that moment — not just a list of changed lines.
- Each commit points to its **parent**, forming a history you can walk backwards through.
- Nothing is a commit until you explicitly say so — Git never saves automatically.

The three states

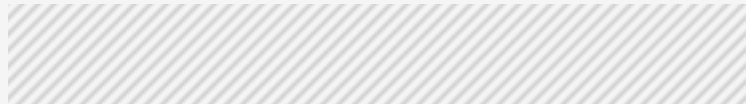


`git add` doesn't save anything permanently — it just marks what you intend to commit next. This two-step process is why you can commit exactly the changes you meant to, not everything you touched.

A commit, edited live

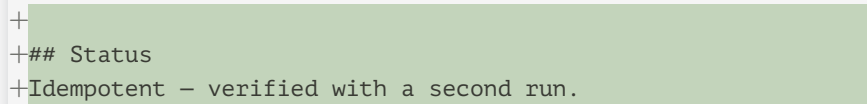
```
# inf345-lab
```

```
A student's Ansible playbook for Lab 02.
```



```
# inf345-lab
```

```
A student's Ansible playbook for Lab 02.
```



```
+  
+## Status  
+Idempotent – verified with a second run.
```

This is what ``git diff`` shows you before you commit: exactly what changed, line by line. Edit either panel — the diff updates live.

The core loop

```
git status  
git add README.md  
git commit -m "Document idempotence check"  
git log --oneline
```

This four-command loop is 90% of what you'll do with Git day to day. Everything else — branches, remotes, rebasing — builds on top of it.

Quick check

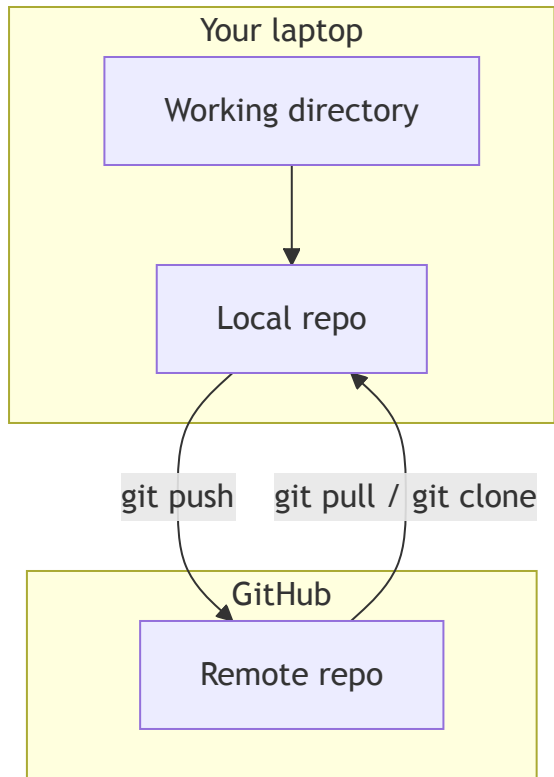
Why does Git separate "staging" from "committing" instead of just committing everything you've changed?

So you can commit a coherent, reviewable unit of change — not an accidental mix of unrelated edits.

Block 2

Branching & the GitHub workflow

Local vs remote



Branches are just pointers

- A branch is a lightweight, movable pointer to a commit — not a copy of the whole project.
- `main` is just a convention for "the default branch," not a special technical concept.
- Creating a branch is instant because it doesn't duplicate any files.

For this course: you generally won't need extra branches — you'll work on `main` in your fork and open a PR. Branches matter more once you're collaborating with others, which is why the capstone uses them.

How you'll actually submit work in this course

```
# 1. Fork the course repo on GitHub (button, top right) – this gives you your own copy
gh repo fork weeebdev/inf345 --clone
cd inf345
# ... do the work in your lab/practice folder ...
git add .
git commit -m "Complete practice exercise"
git push
gh pr create --repo weeebdev/inf345
```

Opening that PR is what triggers the autograder — GitHub Actions checks your PR automatically and posts a score. Your PR isn't meant to be merged; it's just how the check runs. Every GitHub-native lab and practice session in this course follows this loop.

Heads up: forks of a public repo are public too — anyone can see your PR. Fine for practice sessions, not something to worry about copying from.

Block 3

Where to learn more

Curated resources



Pro Git (free book)

git-scm.com/book



GitHub Skills (interactive)

skills.github.com



Learn Git Branching (visual, interactive)

learngitbranching.js.org



Atlassian Git Tutorials

atlassian.com/git/tutorials



Oh Shit, Git!?! (fixing mistakes)

ohshitgit.com

Bookmark these — you'll come back to them all semester, not just today.

Today's practice session — 1 hour

- ☐ Fork `weeebdev/inf345` (button on GitHub, or `gh repo fork`)
- ☐ Make 3 separate commits in `practices/02-git-github/`
- ☐ Open a PR back to `weeebdev/inf345` — that triggers the autograder
- ☐ PR open before the session ends — this is also your attendance signal

See `practices/02-git-github/README.md` for the exact task and definition of done.

Before next lecture



Lesson 3 is Containers 101 — come with these ready:

- ☐ Install Docker Desktop **or** Podman Desktop on your laptop
- ☐ Make sure today's practice PR is open and the autograder is green
- ☐ Skim `labs/01-containers-podman/README.md`

See you next lecture

Containers 101 — why they exist, and how to build your first one.